T^4: Test-Time Training for Taste

Learning Research Reasoning Through Synthetic Literature Review

Sachin Konan

1 Motivation

Scientific discovery requires two capabilities: *solving* problems, and *identifying* which problems are worth solving. Recent progress on the first—long chain-of-thought, generator-verifier paradigms, code agents—is real, but also feels like it yields to scale: more compute and longer inference budgets steadily improve problem-solving on tasks humans can verify. The second capability, *research taste*, is harder. The best researchers pick problems and connect ideas in ways that define fields, sometimes before anyone else sees the importance. Standard pretraining learns $P(\text{paper} \mid \text{literature})$—the distribution over outcomes—but not the *process*: questions asked while reading related work, gaps noticed between lines of work, and why certain gaps were judged worth pursuing.

Recent synthetic data methods give us the tools. Yang et al. (2024) show that knowledge rearrangement—synthesizing diverse representations of facts via entity-graph traversal—dramatically outperforms naive continued pretraining (1.3M→455M tokens, QA accuracy 39.5%→56.2%). Yang et al. (2025) scale this to general pretraining by modeling $P(d_2 \mid d_1)$ across document pairs, recovering 58% of a 20×-data oracle at 1T tokens. Kim et al. (2025) show that organizing synthetic data into long megadocuments yields 1.80× data efficiency, with gains substantially larger on long-context evaluation. Dong et al. (2024) demonstrate scientific evolution is modelable: chronological BERT on 1.75M arXiv papers achieves 0.991 AUC on citation prediction with smooth temporal weight trajectories.

The gap: these methods improve knowledge acquisition or predict citations, but none capture the intermediate reasoning of literature review. We want to synthesize *that process* as training data.

2 Approach: AutoReview Pipeline

We reconstruct research reasoning trajectories from arXiv and assemble them into long-context synthetic training data in three stages.

Stage 1: LLM-Guided Citation Crawl. We use the Kaggle arXiv metadata corpus for title→ID resolution and download per-paper LaTeX source from `arxiv.org/src/{id}`, extracting structured references via BibTeX/BBL parsing. The crawl is BFS over the citation graph: at each node, an LLM ranks references by importance with rationale, and we follow the top-K that have arXiv source. This produces a citation tree where edges carry explicit signals about *why* to follow each citation.

Stage 2: Synthetic Annotation. For each paper, we generate structured CoT annotations *in context*—the LLM sees accumulated reading history: (a) *summary*—core problem, method, result; (b) *gap analysis*—open questions and challengeable assumptions; (c) *importance judgment*—which gaps would be meaningful contributions; (d) *cross-paper connections*—links between gaps/methods/findings across papers not made explicit individually. This is EntiGraph-style knowledge rearrangement, but over scientific judgment rather than domain facts.

Stage 3: Long-Context Megadoc Assembly. Following Kim et al. (2025), we compose traversal traces into long training sequences: paper text → intermediate CoT → gap annotations → cross-paper reasoning → research question formulation. The target is the final research direction; intermediate annotations provide process supervision. These are natural megadocs with latent-thought insertions between segments—exactly the structure yielding the largest long-context efficiency gains.

3 Training

Continued pretraining on assembled megadoc sequences. A single example spans many papers with accumulated reasoning over hundreds of thousands of tokens. Test-time training is a natural fit: the model's understanding should evolve *as it reads*, updating representations on the fly—exactly the setting where TTT methods excel. Hence T^4: test-time training for taste.

The primary training objective is cross-entropy loss on the target tokens: given the full literature review trace (papers, intermediate CoT, gap analyses), predict the final research direction. Intermediate annotations also serve as auxiliary prediction targets, providing process supervision at each step of the reasoning chain.

4 Evaluation

Evaluating research taste is hard—it requires judging the quality of *questions*, not answers. We propose a hierarchy of evaluation strategies, from cheap/automatic to expensive/definitive.

4.1 Intrinsic Metrics

Cross-entropy on held-out directions. The most direct metric: given a literature review context, measure CE loss on the actual research direction the author pursued. We construct held-out sets by taking real papers, reconstructing their citation context from our crawl data, and using the paper's actual contribution as the target. Lower CE means the model assigns higher probability to directions humans actually found worth pursuing.

Auxiliary tasks. We evaluate on tasks that probe components of research taste independently: (a) *retrospective citation prediction*—hold out one reference from a paper's bibliography, predict which paper is missing and why it's relevant; (b) *gap identification*—given papers from a subfield at year Y, predict research directions actually pursued in $Y{+}1$ or $Y{+}2$; (c) *importance ranking*—given a set of open questions extracted from recent papers, rank them by how much follow-up work they actually generated.

4.2 LLM-as-Judge

We use a strong LLM (e.g., Claude, GPT-4) to evaluate generated research directions on: (a) *novelty*—does this direction differ meaningfully from existing work? (b) *specificity*—is this concrete enough to act on, or is it vague hand-waving? (c) *grounding*—does the direction follow logically from the gaps identified in the literature? (d) *feasibility*—could a competent researcher actually execute this? This gives us a scalable proxy for expert judgment that we can run on thousands of generated directions.

4.3 End-to-End: Generate → Execute → Judge

The strongest evaluation closes the loop entirely. Given a generated research direction:

1. **Generate**: the model produces a concrete research idea with methodology, expected results, and contribution statement—essentially a mini-proposal.
2. **Execute**: a code agent (e.g., AI Scientist-style) attempts to implement the idea—runs experiments, generates figures, writes up results.
3. **Judge**: we feed the resulting paper through a *conference acceptance model*—a classifier trained on accept/reject decisions from real venues. This gives us a scalar reward signal: did the generated direction lead to work that looks like it would be accepted?

This creates a generator-verifier loop for research taste. The acceptance model serves as a learned verifier that captures the implicit standards of the research community. Crucially, this evaluates the *full pipeline*—not just whether the model can identify interesting gaps, but whether those gaps lead to executable, publishable contributions. We can also use the acceptance model's score as a reward signal for RLHF-style fine-tuning, iteratively improving the model's taste by optimizing for directions that produce high-quality papers.

4.4 Human Expert Evaluation

As a final check, we run blind comparisons: present domain experts with research directions generated by T^4 alongside directions from actual papers (extracted from their introductions/abstracts), and measure (a) whether experts can distinguish generated from real, and (b) comparative ratings on novelty, feasibility, and importance. This is expensive but necessary to validate that automatic metrics correlate with genuine research quality.

References

Dong, J., Lyu, Z., & Ke, Q. (2024). Towards understanding evolution of science through language model series. *arXiv preprint arXiv:2409.09636*.

Kim, K., Kotha, S., Choi, Y., Hashimoto, T., Haber, N., & Liang, P. (2025). Data-efficient pre-training by scaling synthetic megadocs. *arXiv preprint arXiv:2603.18534*.

Yang, Z., Band, N., Li, S., Candès, E., & Hashimoto, T. (2024). Synthetic continued pretraining. *arXiv preprint arXiv:2409.07431*.

Yang, Z., Zhang, A., Liu, H., Hashimoto, T., Candès, E., Wang, C., & Pang, R. (2025). Synthetic bootstrapped pretraining. *arXiv preprint arXiv:2509.15248*.